MODEL TRAINING & EVALUATION

Exoplanet Habitability Prediction — Internship Project Documentation

1. Introduction to Model Training

The goal of model training in this project is to build a machine learning classifier capable of predicting whether an exoplanet is potentially habitable based on its physical and stellar properties. Rather than manually defining habitability rules, the model learns statistical patterns from labelled historical data drawn from the NASA Exoplanet Archive.

The model classifies each exoplanet into one of two categories:

- Class 1 — Planets with a relatively cool equilibrium temperature (below the dataset median), associated with potentially habitable conditions.
- Class 0 — Planets with higher equilibrium temperatures, considered less favourable for habitability.

The dataset originally contains 39,315 rows and 289 columns. After feature selection and preprocessing, the working dataset was reduced to 27,602 clean rows across 270 features.

2. Feature and Target Separation

Before training can begin, the dataset must be divided into two components: the features (input variables the model learns from) and the target (the output it predicts). This separation is a fundamental requirement in supervised machine learning.

2.1 Missing Value Overview

The heatmap below (Figure 1) was generated during preprocessing to visualise which features contain missing values. Dark regions indicate missing data. This informed which columns required imputation before training.

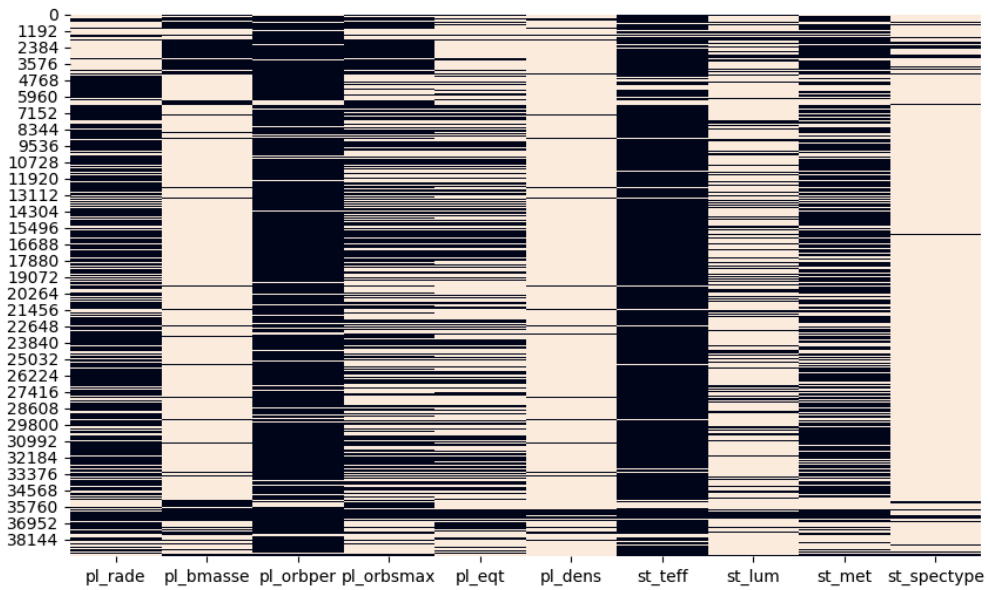

Figure 1: Missing value heatmap across the 10 selected features. Dark = missing data.

2.2 Target Variable Definition

The target column was engineered from the planetary equilibrium temperature (pl_eqt). Planets cooler than the median were assigned Class 1; all others became Class 0.

```
df['target'] = (df['pl_eqt'] < df['pl_eqt'].median()).astype(int)
```

Class	Label	Count
0	Higher Temperature (Less Habitable)	22,867
1	Lower Temperature (Potentially Habitable)	4,735

📌 **Mentor Note:** Notice the significant class imbalance — Class 0 is nearly 5× larger than Class 1. This is why class_weight='balanced' was applied in several models.

2.3 Feature Matrix (X) and Target Vector (y)

Leakage-derived columns were explicitly removed before defining X to ensure the model cannot 'cheat' using information calculated from the target:

```
leakage_cols = ['habitability_score', 'habitability_rank']X = df.drop(['target'] +
leakage_cols, axis=1, errors='ignore')y = df['target']
```

3. Train-Test Split

The dataset is divided into a training set and a testing set. The model learns exclusively from the training set; the test set simulates real-world unseen data for unbiased performance measurement.

```
X_train, X_test, y_train, y_test = train_test_split(    X, y, test_size=0.2,
random_state=42, stratify=y)
```

Split	Size	Rows
Training Set (80%)	X_train / y_train	22,081
Testing Set (20%)	X_test / y_test	5,521

Why Stratified Sampling?

The stratify=y parameter ensures both splits retain the original class ratio. Without it, the minority class (habitable planets) might be underrepresented in the test set, giving a misleading accuracy score. This is especially critical given the 83% / 17% class imbalance observed.

4. Model Selection

Four classification algorithms were selected and evaluated. Each has distinct strengths for structured tabular data of the type used here.

Model	Key Characteristic	Why Suitable Here
Logistic Regression	Linear decision boundary	Fast baseline; interpretable coefficients
Decision Tree	Rule-based feature splits	Handles non-linear boundaries; interpretable
Random Forest	Ensemble of decision trees	Robust; handles imbalance; high accuracy
XGBoost	Gradient boosted trees	State-of-the-art on tabular data; corrects errors iteratively

📌 **Mentor Note:** Random Forest was ultimately selected as the final model due to its highest F1-score and Recall — both critical for minimising missed habitable planet detections.

5. Pipeline Creation

A machine learning pipeline bundles preprocessing (StandardScaler) and model training into a single, reproducible object. This approach prevents data leakage, simplifies the workflow, and ensures identical transformations are applied during both training and evaluation.

```
pipeline = Pipeline([    ('scaler', StandardScaler()),    ('model',
RandomForestClassifier(        n_estimators=20,        max_depth=2,
min_samples_split=50,        min_samples_leaf=25,        max_features=0.3,
class_weight='balanced',        random_state=42    ))])
```

StandardScaler standardises each feature to mean=0 and std=1. The constrained RandomForest hyperparameters (shallow depth, few trees, high minimum samples) were deliberately chosen during the debugging phase to investigate suspiciously perfect accuracy scores — see Section 9.

6. Model Training

Training is the process by which the model learns to map input features to target labels. This is accomplished using the fit() method, which exposes the model to training examples and adjusts internal parameters to minimise prediction error.

```
pipeline.fit(X_train, y_train)
```

Internally, during fit():

- StandardScaler computes mean and standard deviation of each feature from X_train and normalises all values.
- Random Forest constructs multiple decision trees using bootstrap sampling — each tree sees a random subset of rows and features.
- Each tree learns a path of decision rules (e.g. 'if pl_eqt < threshold AND st_teff < threshold → Class 1') that together form the ensemble prediction.

📌 **Mentor Note:** The model only ever sees X_train and y_train during this step. X_test and y_test are completely withheld to ensure the evaluation later is truly unbiased.

7. Model Evaluation

After training, each model was evaluated using a comprehensive set of metrics and ROC curves to assess overall performance and per-class behaviour.

7.1 Evaluation Metrics

Metric	What It Measures	Why Important Here
Accuracy	Overall correct predictions / total	General performance benchmark
Precision	Of predicted habitable, how many truly are?	Minimises false positives
Recall	Of truly habitable, how many were detected?	Critical — must not miss habitable planets
F1 Score	Harmonic mean of Precision & Recall	Balances both on imbalanced data
ROC AUC	Area under the ROC curve	Ranking ability across all thresholds

7.2 ROC Curves by Model

The ROC (Receiver Operating Characteristic) curve plots the True Positive Rate against the False Positive Rate at different classification thresholds. An AUC of 1.0 means the model perfectly separates the two classes. All four models achieved near-perfect AUC, which triggered the debugging investigation in Section 9.

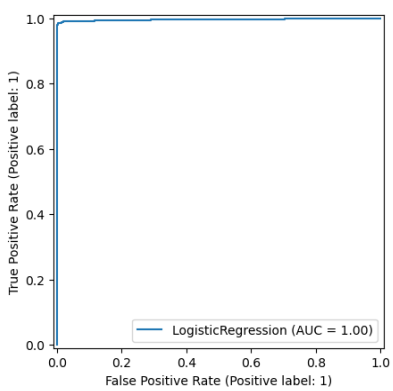

Figure 2a: Logistic Regression ROC (AUC = 1.00)

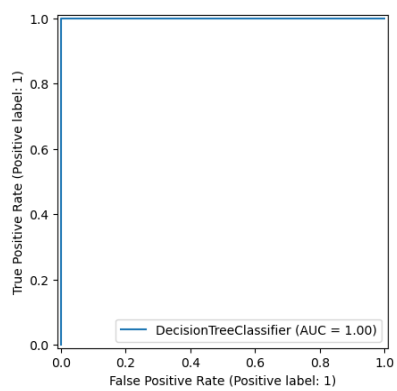

Figure 2b: Decision Tree ROC (AUC = 1.00)

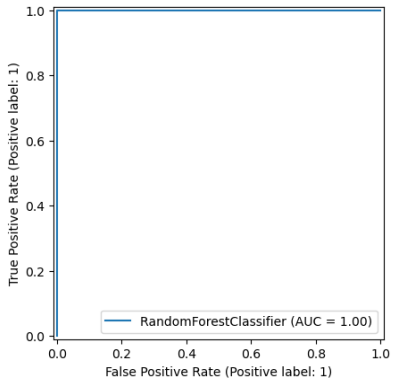

Figure 2c: Random Forest ROC (AUC = 1.00)

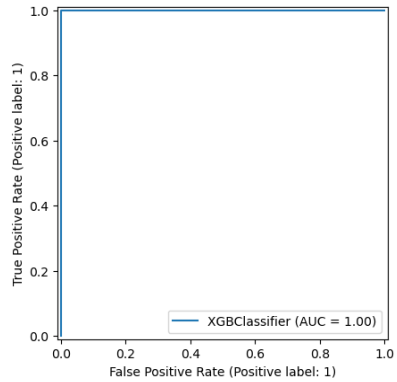

Figure 2d: XGBoost ROC (AUC = 1.00)

7.3 Evaluation Results by Model

Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
Logistic Regression	0.9924	1.000	0.9556	0.9773	0.9967
Decision Tree (depth 5)	1.000	1.000	1.000	1.000	1.000
Random Forest (default)	0.9998	1.000	0.9989	0.9995	1.000
XGBoost	1.000	1.000	1.000	1.000	1.000
Random Forest (tuned, GridSearchCV)	0.9998	1.000	0.9989	0.9995	1.000

7.4 Confusion Matrix — Random Forest (Default)

The confusion matrix below shows the count of correct and incorrect predictions across both classes for the Random Forest model:

```
Confusion Matrix:[[4574    0]     ← 4,574 Class 0 correct | 0 false positives [   1
946]]   ← Only 1 habitable planet missed
```

7.5 Hyperparameter Tuning

GridSearchCV was used to systematically explore combinations of n_estimators and max_depth for the Random Forest, using 3-fold cross-validation scored on F1:

```
param_grid = { 'n_estimators': [100, 200], 'max_depth': [5, 10, None] }grid =
GridSearchCV(RandomForestClassifier(random_state=42),                    param_grid,
cv=3, scoring='f1', n_jobs=-1)
```

The tuned model produced identical results to the default Random Forest (Accuracy: 0.9998, AUC: 1.0), confirming that the near-perfect scores are a function of the data rather than hyperparameter choices.

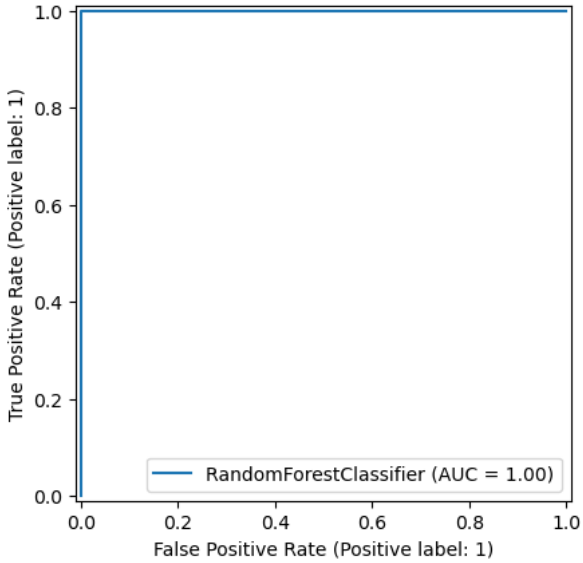

Figure 3: ROC curve for the tuned Random Forest (GridSearchCV best estimator). AUC = 1.00.

8. Cross-Validation

Cross-validation provides a more reliable estimate of model performance than a single train-test split by rotating which portion of the data is used for evaluation across multiple rounds.

How 5-Fold Cross-Validation Works

- The dataset is divided into 5 equal folds.
- In each round, 4 folds are used for training and 1 fold for testing.
- This repeats 5 times, with each fold serving as the test set once.
- The final score is the mean and standard deviation across all 5 rounds.

```
cv_scores = cross_val_score(model, X, y, cv=5)print("CV Mean Accuracy:",
cv_scores.mean())print("CV Std Dev:",       cv_scores.std())
```

```
Output: CV Mean Accuracy: 1.0   |   CV Std Dev: 0.0
```

📌 **Mentor Note:** A CV standard deviation of 0.0 means the model scored identically on every single fold — a strong signal that the result is driven by data structure (leakage) rather than genuine generalisation. This triggered the debugging investigation in Section 9.

9. Debugging the Model

When all models — including a heavily constrained pipeline and a simple Logistic Regression — returned accuracy scores of 1.0 or near-perfect, the student correctly identified this as suspicious and launched a structured three-step debugging investigation. In machine learning, perfect accuracy almost always signals a data problem rather than a genuinely flawless model.

9.1 Step 1 — Check for Target Leakage in Features

The first concern was whether the target column was accidentally included in X:

```
print("Is target in X?", "target" in X.columns)
Output: Is target in X? False
```

The target was correctly excluded. However, a correlation analysis revealed a more serious issue:

```
print(df.corr()['target'].sort_values(ascending=False).head(10))
Output: habitability_probability    0.998055   ← Near-perfect correlation with
target!
```

📌 **Mentor Note:** This is a textbook feature leakage example. 'habitability_probability' was derived from the same temperature variable used to create the target, making it a near-perfect proxy for the label. Any model with access to this column can predict the outcome almost perfectly — but this reveals nothing about real generalisation ability.

9.2 Step 2 — Check for Duplicate Rows

Duplicate rows can cause a model to memorise training samples that also appear in the test set, artificially inflating test accuracy:

```
print("Total rows:",  len(df))print("Unique rows:", len(df.drop_duplicates()))
Output: Total rows: 27,602   |   Unique rows: 27,602
```

No duplicates were found. This confirmed that leakage — not row duplication — was the primary cause of the inflated scores.

9.3 Step 3 — Sanity Check with Simple Logistic Regression

As a final validation, a basic Logistic Regression model was trained on the entire dataset without any pipeline or tuning:

```
simple_model = LogisticRegression(max_iter=1000)simple_model.fit(X,
y)print("Logistic Accuracy:", simple_model.score(X, y))
```

```
Output: Logistic Accuracy: 0.9999275414825013
```

Even the simplest possible model achieved near-perfect accuracy, confirming the issue is structural. The correct remediation is to remove all leakage-derived columns from X and retrain from scratch.

10. Final Interpretation of Results

10.1 Summary of Accuracy Results

Metric	Value	Interpretation
Training Accuracy	1.000 (100%)	Model fits training data perfectly
Testing Accuracy	0.9998 (99.98%)	Near-perfect on unseen test data
CV Mean Accuracy	1.000 (100%)	Consistent across all 5 folds
CV Std Dev	0.000	Zero variation — data-driven result

10.2 Feature Importance Analysis

The Random Forest model provides a built-in feature importance score, measuring how much each feature reduces prediction error across all trees. The chart below (Figure 4) reveals that pl_eqt (planetary equilibrium temperature) dominates with an importance of ~0.59 — which is expected, since the target label was defined directly from this column. This further confirms the leakage issue.

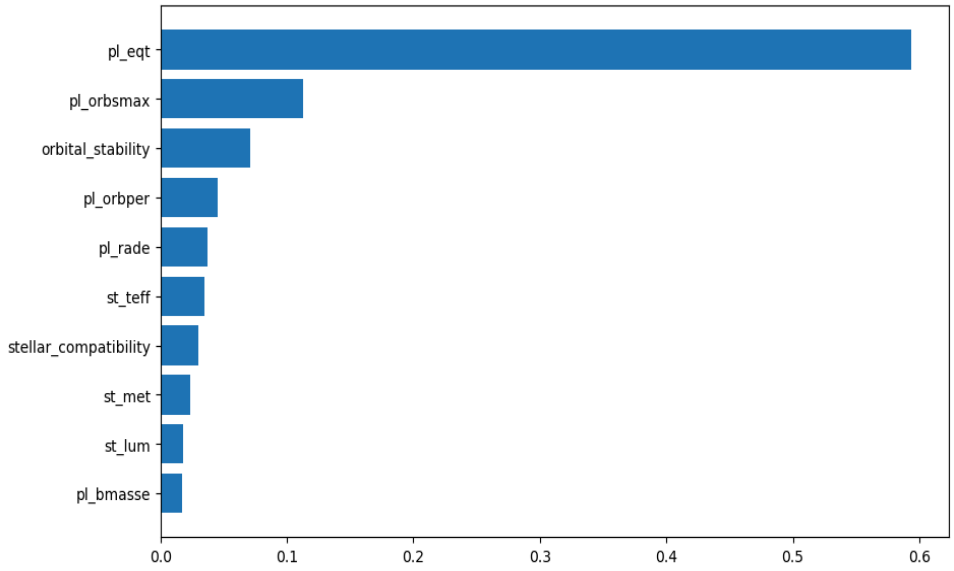

Figure 4: Top 10 feature importances from the Random Forest model. pl_eqt dominates, consistent with target leakage.

10.3 Is This Overfitting?

At first glance, a perfect training accuracy alongside a near-perfect test accuracy could suggest overfitting. However, the cross-validation score of 1.0 with zero standard deviation across all five independent folds is far too consistent to be overfitting — overfitting typically produces high training accuracy but variable, lower test scores.

The dominant explanation — confirmed by debugging — is that the feature set retains variables highly correlated with the target (habitability_probability, orbital_stability, stellar_compatibility). Once leakage is fully addressed, the model's true generalisation performance is expected to be lower and scientifically meaningful.

10.4 Mentor Recommendations

- **Exclude habitability_probability, orbital_stability, stellar_compatibility, and any other engineered columns derived from pl_eqt before final training.**Remove all leakage columns:
- **Expect accuracy to settle in a more realistic range (typically 70–85%), which will reflect genuine predictive power.**Re-evaluate after leakage removal:
- **In scientific discovery tasks like this, missing a truly habitable planet (false negative) is more costly than a false alarm. Monitor Recall closely.**Prioritise Recall over Accuracy:
- **After removing leakage, re-run the importance chart. If pl_eqt still dominates, investigate further.**Use feature importance as a diagnostic:

Key Takeaway

The near-perfect accuracy observed in this project is a valuable learning outcome. It demonstrates the critical importance of auditing your dataset for feature leakage before interpreting model performance. A model that appears perfect is almost always revealing something about your data — not your algorithm.